# ProactiveHop: Graph-Structured Reinforcement Learning for Proactive Session Migration in Game Server Clusters

Subodh Wandile[a]

[a] *Independent Researcher*

**Abstract**

Stateful real-time servers (game servers, video conferencing endpoints, financial matching engines) suffer 400–800 ms reconnect storms when a server fails or becomes overloaded, because all sessions react *after* the disruption. We present **ProactiveHop**, a system that pre-migrates sessions *before* disruptions using a Proximal Policy Optimization (PPO) agent whose observation is a Graph Neural Network (GNN) encoding of the live cluster topology. The GNN is designed to capture structural signals—rising failure probability on a neighbor, correlated load spikes propagating along fat-tree links—that are invisible to per-server threshold heuristics.

We implement ProactiveHop on a discrete-event simulator of a $k$=4 fat-tree cluster and evaluate against No-Migration, Threshold-70%, LSTM Predictor, PPO+MLP, and a $p_{\text{drop}}$-threshold heuristic. Key results: (1) in a controlled server-crash scenario (20 seeds), PPO+GNN proactively migrates 86.0%±31.8% of affected sessions before the crash (median 100%)—significantly exceeding Threshold-70% (26.6%±26.3%, Wilcoxon signed-rank $p<0.001$, Holm-corrected, 3.2×); PPO+MLP achieves the highest crash-scenario rate (98.0%±6.1%, $p$=0.14 vs GNN); GNN is not significantly different from LSTM (54.8%±41.4%, $p$=0.023, n.s. after Holm); no pairwise difference among learned policies (GNN/MLP/LSTM) reaches significance at $n$=20; (2) in a multi-seed i.i.d. ablation (5 training seeds × 20 held-out seeds), GNN achieves **1.8× higher catch rate** (1.64% vs. 0.90%, Welch $t$=5.53, $p$=0.0006) than MLP under identical PPO training, confirmed by Mann-Whitney ($p$=0.004 one-sided); edge-removal ablation (GNN-NoEdge-eval $p$=0.012) shows the trained model relies on its edge inputs at evaluation time, though an edge-free GNN trained from scratch matches performance ($p$=0.78); (3) inference time grows only **1.1**× when cluster size grows **4**× ($N$=16 → 64) vs. 4.3× for LSTM, verifying the

near-linear inference scaling of local neighbourhood aggregation.

ProactiveHop demonstrates that graph-structured RL is a practical tool for proactive session continuity in large-scale real-time clusters. In the crash scenario, GNN pre-migrates ≈86% of sessions vs. ≈27% for Threshold, a 3.2× improvement (Wilcoxon $p<0.001$, Holm-corrected); PPO+MLP achieves the highest crash-scenario rate (98%) though the difference from GNN is not significant ($p=0.14$); the primary proven GNN-vs-MLP advantage is a 1.8× higher i.i.d. catch rate, combined with near-linear inference scaling and zero-shot generalization to larger clusters.

*Keywords:* graph neural networks, reinforcement learning, proactive session migration, server cluster management, proximal policy optimization, fat-tree topology, load-coupled failure prediction

---

## 1. Introduction

Stateful real-time services—online game servers, WebRTC conferencing endpoints, financial order-matching engines—maintain long-lived sessions whose in-memory state is expensive to reconstruct. When a server fails or saturates its CPU, sessions disconnect and must reconnect reactively. The disruption window is well-documented across production systems:

- **Game servers (AWS GameLift, Agones):** AWS explicitly states that "game sessions actively running on a reclaimed instance are *interrupted*" [1]; the Agones fleet manager *deletes* unhealthy servers and spawns replacements, with a minimum detection latency of 3 × `periodSeconds` (default 15 s) before any reconnect can begin [2].
- **WebRTC / video conferencing (Jitsi, RFC 8445):** When the underlying server fails, an in-place ICE restart cannot reuse the dead transport; a full new `PeerConnection` with fresh DTLS/SRTP negotiation takes 1–3 s [3]. Applications must trigger this manually—"the browser will not trigger one on its own" [4]. Jitsi's

*Email address:* subodhwandile10@gmail.com (Subodh Wandile)

GitHub pull request #2438 shows in-place SFU failover is being addressed but not yet standard [5].

- **Routing latency (Riot Games):** Riot built an entire private backbone (Riot Direct) specifically to avoid reactive path failures that pushed round-trip latency from 14 ms to 70 ms—a 5× spike—on commodity routing [6].

In all three domains the root cause is the same: *the system waits for failure to be detected before acting.* Detection introduces multi-second delays; reconnect adds 1–3 s more. For game servers and video calls this is a hard user-visible disruption.

Existing mitigation approaches fall into three categories:

1. **No migration**: sessions reconnect after failure. Simple but reactive; inherits the full detection + reconnect delay.
2. **Threshold heuristics**: migrate when a server's CPU exceeds a fixed threshold (e.g., 70%). Acts before failure only if load spiked first; blind to failure-rate signals and neighbor state.
3. **Hot standby**: keep a replica server warm (e.g., AWS RDS Multi-AZ). Eliminates reconnect but doubles resource cost and still requires detection before switchover.

None of these approaches uses *structural topology signals.* A fat-tree cluster has a rich graph structure: a server with rising failure rate and two neighbors already at 80% load is qualitatively different from the same server with idle neighbors. Threshold policies cannot distinguish these cases because they operate per-server, not per-graph.

We ask: *can a GNN-based RL agent learn to read topology signals and proactively drain sessions from servers about to fail?*

**Contributions.**

- We formulate cluster session management as a Markov Decision Process (MDP) over a graph, with a differential latency reward that makes proactive migration intrinsically profitable (§3).

- We design a two-layer GraphSAGE encoder that processes only the 1-hop neighbourhood of each node, with per-layer cost $O(|E| \cdot d_h + N \cdot d_h^2)$—linear in $N$ for bounded degree (§4).

Table 1: Comparison of session/task migration and RL networking approaches. "Proactive" = acts before failure/overload; "Topo-aware" = uses graph structure; "Stateful" = migrates session state with reconnect cost.

| System | Proactive | Topo-aware | RL | Stateful | Simulated |
|---|---|---|---|---|---|
| Staying Alive [7] | × | × | × | ✓ | × |
| Decima [8] | ✓ | ✓ | ✓ | × | ✓ |
| WLAN Handoff [9] | ✓ | ✓ | ✓ | × | ✓ |
| Pensieve [10] | × | × | ✓ | × | ✓ |
| AuTO [11] | ✓ | × | ✓ | × | × |
| LiveMigration [12] | ✓ | × | × | ✓ | × |
| **ProactiveHop (ours)** | ✓ | ✓ | ✓ | ✓ | ✓ |

- We implement ProactiveHop on a fat-tree simulator with Weibull failure injection, delta-compressed session state, and correlated load spikes (§5).
- We run controlled experiments showing PPO+GNN achieves **1.8× higher catch rate than MLP** (1.64% vs 0.90%, $p$=0.0006, Welch $t$-test, 5 training seeds × 20 held-out seeds) in i.i.d. evaluation, where the gain is attributable to the shared-weight GNN encoder rather than to topology (edge-removal ablation $p$=0.78 when trained from scratch); scales near-linearly to $N$=64 servers zero-shot (inference 1.1× vs 4.3× for LSTM); and in a controlled crash scenario (20 evaluation seeds) GNN (86.0%) significantly outperforms Threshold-70% (26.6%, Wilcoxon $p$<0.001, Holm-corrected, 3.2×); PPO+MLP achieves the highest crash-scenario rate (98.0%, $p$=0.14 vs GNN); GNN is not significantly different from LSTM (54.8%, $p$=0.023, n.s. after Holm); no pairwise difference among learned policies significant at $n$=20 (§6).

## 2. Related Work

Table 1 positions ProactiveHop against the most closely related work across six dimensions.

*Reactive session migration..* "Staying Alive" [7] reselects connection paths at the network edge when the current path degrades, keeping sessions alive without full reconnect. Migration is *triggered reactively*, inheriting the full 400–800 ms reconnect latency, since the system must wait for the failure to be confirmed before acting. Clark et al.'s live migration [12] pre-copies VM memory pages while the VM is running (warm migration) to minimize downtime, but the *trigger* remains a manual operator command or a reactive health check. Neither system uses predictive signals or graph topology to act before symptoms become failures. ProactiveHop targets the window between first observable symptom (rising $\lambda_v$, CPU stress) and confirmed failure.

*GNN-based cluster scheduling..* Decima [8] applies GNN+RL to Spark job scheduling, where the computation graph (job DAG) is the structured input. The GNN aggregates stage-to-stage dependencies; the action is assigning stages to executors. Our graph is the *physical cluster topology*, not a job DAG: nodes are servers, edges are network links with latency and bandwidth attributes, and the action is a binary per-server pre-migration trigger. The key difference is that Decima's graph is given (the submitted job), whereas ProactiveHop must reason about *evolving* load and failure signals on a fixed physical topology. AuTO [11] uses a two-level RL system for flow scheduling in data centres, but operates at the flow level without session state and without graph-structured encoding.

*Handoff via RL in dense WLANs..* A substantial line of work uses RL for access-point handoff in dense wireless LANs [9]. These approaches model the problem as a Markov decision process where a mobile client moves through a graph of fixed base stations; the policy decides when to hand off to a neighbouring cell. The graph topology is the radio access network, and the state is signal strength and cell load. Our setting is the *inverse*: clients are stationary (game sessions, WebRTC peers) and servers can fail or become overloaded. Migration is a *business* decision with an explicit bandwidth cost (session state transfer), not a physical necessity imposed by client movement. Furthermore, handoff systems operate per-UE on a single radio link; ProactiveHop operates over a full fat-tree cluster with $N$=16–64 servers simultaneously.

*RL for networking..* Pensieve [10] applies deep RL to adaptive bitrate (ABR) video streaming, learning a per-chunk bitrate policy from throughput history. CPC [13] uses

contrastive predictive coding to learn latent representations of sequential data. Both operate on a *single* client–server link with scalar state; neither has a graph topology or a migration cost. ProactiveHop extends the RL-for-networking line to *multi-server graph-structured* decisions, where the observation is a GNN embedding of the live cluster graph and the reward explicitly penalises unnecessary session state transfers.

*Proactive resource management..* Lin et al. [14] predict node failure in cloud service systems from runtime performance metrics (CPU, memory, network counters) and trigger migration before the predicted failure window. The key differences are (i) their approach uses a per-node time-series predictor on performance counters rather than a graph-structured policy over the cluster; (ii) it migrates full VMs with operator-confirmed triggers, not real-time sessions with sub-second reconnect budgets; and (iii) it does not jointly optimise migration decisions across servers. ProactiveHop combines the predictive insight of Lin et al. with the topology reasoning of Decima in a single end-to-end RL policy.

## 3. Problem Formulation

### *3.1. System Model*

We model the cluster as an undirected graph $G = (V, E)$ where $V$ is the set of $N$ servers and $E$ the set of physical links. Each server $v \in V$ at time step $t$ has a feature vector:

$$\mathbf{x}_v^t = \big[\, u_v,\ m_v,\ \sigma_v,\ \ell_v,\ p_v^{\text{drop}},\ a_v,\ \lambda_v \,\big] \in \mathbb{R}^7$$

where $u_v$ = CPU utilization, $m_v$ = memory utilization, $\sigma_v$ = session ratio (sessions / capacity), $\ell_v$ = composite load, $p_v^{\text{drop}} = \sigma\big((\ell_v - 0.72) \cdot 15\big)$ = drop probability (steep sigmoid for clean gradient), $a_v \in \{0, 1\}$ = alive flag, and $\lambda_v$ = load-coupled failure hazard rate, $\lambda_v = 20 \cdot \big[p_{\text{base}} + 0.005 \cdot \max(0,\ \ell_v - 0.70)\big]$, where $p_{\text{base}} = 1 - e^{-(\Delta t/\text{MTBF})^{1.5}}$ is the baseline Weibull per-step probability. Note that $\lambda_v$ is a deterministic function of $\ell_v$ plus the fixed MTBF baseline; it provides the GNN a *scaled and thresholded* view of overload risk distinct from the raw $\ell_v$ value.

Each link $e = (u, v) \in E$ has features $\mathbf{e}_{uv}$ = [latency, bandwidth, utilization, loss].

*3.2. MDP Definition*

**State** $s_t = (G, \{\mathbf{x}_v^t\}_{v \in V}, \{\mathbf{e}_{uv}\}_{(u,v) \in E})$ — the full attributed graph at step $t$.

**Action** $\mathbf{a}_t \in \{0,1\}^N$: a binary flag per server. $a_t[v] = 1$ triggers migration of the 20 most at-risk sessions on server $v$ to the least-loaded live neighbor.

**Transition**: the simulator applies *persistent* migration decisions—sessions move immediately to the target server. At step end, a *counterfactual* drop check runs at the original server: if the server's drop probability would have fired, the migration is a true positive (TP); otherwise it is a false positive (FP). The simulator also advances arrivals and departures, injects correlated load spikes (probability 0.08 per step), and samples load-coupled Weibull failures (shape $k$=1.5, scale MTBF; see $\lambda_v$ above).

**Reward.** Let $m_t$ be the step metrics tuple. We use a *differential latency reward*:

$$r_t = m_t^{\mathrm{TP}} \cdot \left(\bar{L}_{\mathrm{react}} - \bar{L}_{\mathrm{spec}}\right) - C_{\mathrm{bw}} \cdot B_t \tag{1}$$

where $m_t^{\mathrm{TP}}$ = true-positive migrations (sessions moved proactively to a server whose old host would have dropped them), $\bar{L}_{\mathrm{react}} = 450\,\mathrm{ms}$ = mean reactive reconnect latency, $\bar{L}_{\mathrm{spec}} = 5\,\mathrm{ms}$ = mean proactive handoff latency (live session transfer cost), $C_{\mathrm{bw}} = 5 \times 10^{-5}$ = bandwidth penalty coefficient, and $B_t$ = migration bandwidth in KB.

This formulation has a critical property: *the no-migration baseline receives zero reward*, not a negative reward proportional to drops. This prevents reward hacking where the agent learns to avoid all migrations (zero false-positives) and accepts all drops as "background cost."

*3.3. Representational Separation: Threshold vs. 1-hop GNN*

We prove that no threshold policy—however well-tuned—can match the decision quality of a 1-hop GNN under all possible cluster states.

**Definition 1.** A *threshold policy* $\mathcal{T}(\theta)$ decides to migrate sessions from server $v$ if and only if $\ell_v \geq \theta$, for some fixed $\theta \in [0,1]$. It is a function solely of $\ell_v$.

**Proposition 1.** There exist two cluster states $S_1, S_2$ and a server $v$ such that:

1. $\ell_v(S_1) = \ell_v(S_2)$ (identical local load),
2. The 1-hop GNN's representation of $v$ differs between $S_1$ and $S_2$, so the actor head *can* produce different migration decisions,

3. Any threshold policy $\mathcal{T}(\theta)$ produces the *same* decision in $S_1$ and $S_2$,
4. No fixed threshold can produce both decisions; the GNN has the representational capacity to do so.

*Proof.* Let $\ell_v = 0.72$ in both states.

**State** $S_1$: All neighbors $u \in \mathcal{N}(v)$ have load $\ell_u = 0.3$. Migration sends sessions to lightly loaded neighbors.

**State** $S_2$: All neighbors have $\ell_u = 0.85$. All available targets are saturated, so migration is likely to cause unnecessary bandwidth cost with limited benefit.

Any $\mathcal{T}(\theta)$ with $\theta \leq 0.72$ migrates in both states; with $\theta > 0.72$ it migrates in neither. No fixed $\theta$ can make different decisions in $S_1$ and $S_2$.

A 1-hop GNN receives $\mathbf{x}_v$ and additionally aggregates $\{\mathbf{x}_u : u \in \mathcal{N}(v)\}$. Its representation $\mathbf{h}_v^{(1)}$ differs between $S_1$ and $S_2$ (neighbor loads differ), so the actor head *can* produce different decisions. □

**Remark 1.** Under the persistent migration mechanic, a migrated session's TP classification is determined by the *old* server's counterfactual drop probability. For $\ell_v = 0.72$ (the proof's baseline), the per-step counterfactual TP rate is $p_{\text{drop}}(0.72) = 0.5$. The proposition therefore establishes a *representational* result: the GNN can encode a decision boundary in $(\ell_v, \bar{\ell}_{\mathcal{N}(v)})$ space that no scalar threshold can. Whether the RL optimisation finds that boundary under the one-step counterfactual reward is an empirical question (§6.3), not a consequence of this proposition.

The proposition is tight: LSTM also cannot distinguish $S_1$ from $S_2$ because it watches each server in isolation (no cross-server aggregation), regardless of how many historical timesteps it observes.

## 4. Model Architecture

### *4.1. GNN Encoder*

We use a two-layer GraphSAGE [15] encoder. The forward pass has four stages.

*Edge-feature injection..* Each directed edge $(v, u)$ carries a feature vector $\mathbf{e}_{vu} \in \mathbb{R}^4$ (link latency, bandwidth, utilization, packet-loss). Before graph convolution, per-edge context is projected and sum-aggregated into source nodes:

$$\tilde{\mathbf{x}}_v = \mathbf{x}_v + \sum_{(v,u)\in\mathcal{E}_v} W_e\, \mathbf{e}_{vu}, \qquad W_e \in \mathbb{R}^{7\times 4} \tag{2}$$

where $\mathcal{E}_v$ is the set of edges incident to $v$.

*Input projection..* The 7-dimensional augmented features are lifted to the hidden dimension:

$$\mathbf{h}_v^{(0)} = \mathrm{ReLU}\big(W_{\text{in}}\, \tilde{\mathbf{x}}_v\big), \qquad W_{\text{in}} \in \mathbb{R}^{d_h\times 7},\ d_h = 64 \tag{3}$$

*GraphSAGE layers ($l = 1, 2$)..* Each layer performs neighborhood aggregation followed by a residual update:

$$\tilde{\mathbf{h}}_v^{(l)} = \mathrm{ReLU}\Big(W_l \cdot \big[\mathbf{h}_v^{(l-1)} \,\|\, \mathrm{MEAN}_{u\in\mathcal{N}(v)} \mathbf{h}_u^{(l-1)}\big]\Big) \tag{4}$$

$$\mathbf{h}_v^{(l)} = \mathbf{h}_v^{(l-1)} + \mathrm{Dropout}\big(\tilde{\mathbf{h}}_v^{(l)},\, p{=}0.1\big) \tag{5}$$

where $W_l \in \mathbb{R}^{d_h\times 2d_h}$ and all tensors remain in $\mathbb{R}^{d_h}$—the input projection ensures no dimension mismatch.

*Normalization and output projection..* After both layers, a single Layer Normalization followed by a linear output projection yields the final node embeddings:

$$\mathbf{z}_v = W_{\text{out}}\, \mathrm{LN}\big(\mathbf{h}_v^{(2)}\big), \qquad W_{\text{out}} \in \mathbb{R}^{d_h\times d_h} \tag{6}$$

Hidden dimension $d_h = 64$, output dimension $d_h = 64$.

Each layer aggregates only the 1-hop neighborhood. Two layers therefore aggregate the 2-hop neighborhood. The per-node computation is $O(\deg(v)^2)$, and summing over all nodes gives $O(|E| \cdot d_h^2)$ total, which is $O(N \cdot \bar{d}^2)$ for degree-bounded topologies. Fat-tree degree is bounded by $k$ (pod size), so inference time is $O(N \cdot k^2)$ for fixed $k$. Because $k$ is a topology constant (4 in our fat-tree) while $N$ grows with cluster size, latency grows linearly in $N$ in the worst case but empirically near-linearly (1.1× for a 4× increase in $N$) as shown in §6.4.

*4.2. Actor-Critic Network*

The GNN produces node embeddings $\{\mathbf{h}_v\}$. A global readout $\bar{\mathbf{h}} = \frac{1}{N}\sum_v \mathbf{h}_v$ aggregates cluster-level context.

**Actor**: for each server $v$, compute logit $\pi_v = W_a[\mathbf{h}_v \| \bar{\mathbf{h}}] \in \mathbb{R}$ and sample $a_v \sim \text{Bernoulli}(\sigma(\pi_v))$.

**Critic**: $V(s) = W_c\bar{\mathbf{h}} \in \mathbb{R}$.

Both $W_a, W_c$ are two-layer MLP heads (hidden size 64, ReLU) applied after the shared GNN.

*4.3. PPO Training*

We train with Proximal Policy Optimization [16] with GAE ($\lambda = 0.95$), clip $\epsilon = 0.2$, entropy coefficient 0.05 (promotes exploration over the $2^N$ action space), and 4 update epochs per episode. Episode length is 100 steps; training runs for 500 episodes. All experiments use CPU inference. Full hyperparameters are listed in Table A.7.

## 5. Simulator

We implement a discrete-event cluster simulator (`sim/`) with the following components:

**Topology**: fat-tree with $k = 4$ (16 servers). All paper experiments use this topology; zero-shot transfer to larger clusters ($N$=25; $N$=64) is evaluated in §6.4.

**Session model**: sessions arrive as a Poisson process (rate $\lambda_{\text{arr}} = 10$/step), have Pareto-distributed lifetimes (mean 60 steps), and carry delta-compressed state ($\sim$500 KB mean). Persistent pre-migration costs $\bar{L}_{\text{spec}} = 5$ ms (fast session state transfer); reactive reconnect costs $\bar{L}_{\text{react}} \sim \mathcal{N}(400, 150^2)$ ms.

**Failure model**: each server fails per step with probability $p_f = 1 - \exp\big(-(\Delta t/\text{MTBF})^{1.5}\big)$ (Weibull shape 1.5). Failed servers recover after Exp(MTTR) steps. Drop probability on overloaded *live* servers follows $p_{\text{drop}}(v) = \sigma\big((\ell_v - 0.72) \times 15\big)$.

**Load dynamics**: loads decay toward a base load $\ell_0 = 0.55$ with damping 0.05 per step, plus correlated spikes (probability 0.08, magnitude 0.35) that persist for 10 steps and spread to neighboring servers.

*5.1. Simulator Fidelity Validation*

A common critique of simulation-based evaluation is that the simulator may not reproduce the statistical properties of real deployments. We validate three key properties against empirical data and theoretical expectations.

*Failure inter-arrival distribution..* The simulator draws per-step failure probability from a Weibull distribution with shape $\kappa$=1.5 and scale $\eta$=3600 s (MTBF 3600 s). Shape $\kappa$=1.5 was chosen to match empirical hazard rates reported in field studies of disk and DRAM failure [17]. Note that the simulator applies the Weibull CDF as an *i.i.d. per-step* probability—each timestep draws a fresh Bernoulli sample at the same fixed hazard rate—rather than as a true aging process across steps. The per-step probability $p_f = 1 - e^{-(\Delta t/\eta)^\kappa}$ evaluates to $\approx 4.6 \times 10^{-6}$ per server per second, corresponding to roughly one server failure every ≈60 hours *per server*, or one cluster-level failure every ≈3.8 hours across a 16-server cluster—consistent with reported MTBF in hyperscale data centres [17].

*Session arrival process..* Sessions arrive as a Poisson process with rate $\lambda_{\text{arr}}$. The Poisson assumption is standard for modelling independent user connections at the session layer and is validated by the Alibaba 2018 trace experiment (§6.6): under 4.3× higher coefficient of variation (real trace vs. synthetic Poisson), PPO+GNN's catch rate drops only 0.2 percentage points, indicating the policy is robust to the Poisson approximation. Session lifetimes follow a Pareto distribution with shape $\alpha$=1.5, matching the heavy-tailed flow-size distributions observed in data-centre traffic studies [8].

*Load spike propagation..* Correlated load spikes (probability 0.08 per server per step, magnitude 0.35, duration 10 steps) model bursty traffic patterns such as flash crowds or cache invalidation storms. The spike propagates to neighbouring servers with a 50% probability, reflecting intra-cluster load imbalance observed when a hot shard floods the network path to its replicas. With spike probability 0.08 per step and duration 10 steps, servers spend roughly 44% of time in a spike state; the measured peak composite load during spiking (post-warmup, 5 seeds, 500 steps each) reaches $0.92 \pm 0.10$, consistent with heavily-utilised clusters during burst periods [6]. The 44% spiking fraction keeps a

substantial share of servers transiently above the $\ell_v$=0.72 drop threshold; between spikes, load decays toward the base ($\ell_0$=0.55) where drop probability is low.

## 6. Evaluation

### *6.1. Baselines*

- **No-Migration**: sessions reconnect reactively after any drop or failure.
- **Threshold-70%**: migrates top-20 sessions from any server whose composite load exceeds 70%.
- **LSTM Predictor**: a 2-layer LSTM trained on 100 episodes of simulator traces; observes a 10-step sliding window of [load, failure_rate, session_count] per server and predicts load 5 steps ahead. Migrates if predicted load $> 0.65$. This baseline has *temporal memory but no topology.*
- $p_{\text{drop}}$ **heuristic**: thresholds $p_{\text{drop}}(v)$ (feature index 4 in the 7-dim observation vector) at $5 \times 10^{-3}$ and pre-migrates all sessions from any server above threshold. It is a $p_{\text{drop}}$-threshold strategy bound; because it uses the same observable signal as the agent, it shows how much a perfectly tuned greedy policy over that signal could achieve.
- **PPO+GNN (ours)**: trained 500 episodes over 5 random seeds (seeds 0–4); evaluated on held-out seeds. The crash-scenario checkpoint (`best_GNN.pt`) is the highest-reward checkpoint from seed 0; the i.i.d. ablation uses all 5 checkpoints. Policy decisions use greedy (argmax) action selection for deterministic, reproducible evaluation.
- **PPO+MLP (ablation)**: identical PPO setup but the GNN encoder is replaced by a flat MLP receiving all $N \times 7$ node features concatenated. Same training budget, reward, and hyperparameters. Isolates the contribution of graph structure.

### *6.2. Failure Scenario: Controlled Pre-Failure Ramp*

Table 2 shows pre-migration results across 20 evaluation seeds. Server 3 crashes at step 40. From step 30 we inject a *degradation ramp*: CPU utilization rises from 0.35 to 0.95,

Table 2: Pre-migration results: Server 3 crashes at step 40. Exactly 25 sessions seeded on target at step 30 (Server 3 reset to base load before seeding); denominator is identical across all policies. Pre-mig. (%) = fraction of the 25 seeded sessions moved off Server 3 by policy decision during the degradation ramp (steps 30–39), tracked via `_migrated_this_step`. RampMigs = total cluster-wide policy migrations during steps 30–39 (mean±std per seed). Results are mean±population-std over 1 checkpoint × 20 evaluation seeds; PPO agents use greedy (argmax) action selection for deterministic evaluation. CoV = std/mean. Wilcoxon signed-rank (paired), Holm–Bonferroni corrected (4 tests): GNN vs Threshold $p$=0.0001 (survives Holm); GNN vs LSTM $p$=0.023 (n.s. after Holm); GNN vs MLP $p$=0.14; GNN vs $p_{\text{drop}}$ $p$=0.058; no pairwise difference among learned policies (GNN/MLP/LSTM) reaches significance. PPO+MLP uses the same RL framework but a flat MLP encoder (no graph).

| Policy | Pre-mig. (%) | P50 (ms) | P95 (ms) | CoV | RampMigs |
|---|---|---|---|---|---|
| No Migration | $0.0 \pm 0.0$ | 399 | 625 | — | $0 \pm 0$ |
| Threshold-70% | $26.6 \pm 26.3$ | 393 | 629 | 0.99 | $59 \pm 34$ |
| LSTM Predictor | $54.8 \pm 41.4$ | 395 | 632 | 0.76 | $163 \pm 122$ |
| PPO+GNN | $86.0 \pm 31.8$ | 162 | 589 | 0.37 | $2172 \pm 1598$ |
| $p_{\text{drop}}$ heuristic | $99.6 \pm 1.2$ | 10 | 574 | 0.01 | $2535 \pm 1619$ |
| PPO+MLP | $98.0 \pm 6.1$ | 351 | 628 | 0.06 | $1432 \pm 1772$ |

which through the load-coupling formula increases the observable $\lambda_v$ signal, modelling pre-failure symptoms (thermal throttling, disk I/O errors, memory ECC corrections).

**Metric definition**: Pre-mig. (%) counts any policy-driven move of a seeded session off Server 3 during steps 30–39, not whether that move was a correct prediction. All learned policies respond to the rising load signal by migrating sessions; the metric measures how many of the 25 seeded sessions were evacuated before the crash.

All learned policies significantly outperform No Migration (0%) and Threshold-only (26.6%). PPO+MLP achieves the highest crash-scenario rate (98.0%); mean ramp-migration count is lower for MLP than GNN (1432±1772 vs 2172±1598), but variance is high and the difference was not tested, so no efficiency claim is made. GNN is not significantly different from MLP ($p$=0.14, Wilcoxon). **PPO+GNN (86.0%)** significantly outperforms Threshold ($p$<0.001, Wilcoxon, Holm-corrected, 3.2×). GNN is not significantly different from LSTM (54.8%, $p$=0.023, n.s. after Holm). None of the pairwise differences among learned policies reach significance at $n$=20. CoV values: GNN

(0.37), MLP (0.06), LSTM (0.76), Threshold (0.99). MLP shows the most consistent pre-migration rate (lowest CoV); LSTM is most variable. Seed breakdown: GNN is 100% in 13/20 seeds and 0% in 2/20 seeds (bimodal); MLP is 100% in 16/20 seeds with no 0%-seeds; LSTM is 100% in 6/20 and 0% in 4/20.

In the seed-42 representative run, Server 3 starts with approximately 25 seeded sessions (identical denominator across all policies; organic sessions also present but not counted). Server 3's load is reset to base level at seeding to prevent warm-up spikes from pre-draining sessions before the ramp signal appears. The pre-migration count captures only the 25 seeded sessions that were moved off Server 3 by a policy decision during steps 30–39, tracked via `sim._migrated_this_step`. During the degradation ramp (steps 30–39), load on Server 3 rises from 0.35 to 0.95; learned policies respond to this rising signal and drain Server 3 well before the crash at step 40. Threshold-70% (26.6%) can only act once load exceeds 0.70 (≈step 36), leaving a narrower window; learned policies begin acting earlier and achieve substantially higher pre-migration rates (3.2× for GNN vs Threshold). In i.i.d. evaluation without a ramp (Table 3), LSTM achieves 1.58%±0.19% (comparable to GNN's 1.64%, $p$=0.55). The significant GNN vs. MLP gap ($p$=0.0006) persists in i.i.d. evaluation; the crash scenario does not produce significant differences among learned policies at $n$=20.

The GNN encoder's structural advantage is captured mainly in i.i.d. evaluation (Table 3). Threshold-70% (26.6%) acts only on load, missing the earlier action-threshold signal available to trained policies.

Figure 1 plots the session-drain time-series (seed 42).

### *6.3. Ablation: GNN vs. Flat MLP, LSTM, and Edge-Removed GNN*

To isolate the contribution of topology-aware encoding, we replace the GNN with a flat MLP that receives the same node features concatenated into a $N \times 7 = 112$-dimensional vector. The MLP uses three layers (widths $256 \to 128 \to 64$) with the same output dimension (64) and identical PPO hyperparameters. This controls for the RL framework, reward, and training budget.

Table 3 summarises the multi-seed ablation (5 training seeds × 20 held-out seeds).

**GNN vs. MLP.** GNN achieves **1.8× higher catch rate** (1.64% vs. 0.90%; Welch $t$=5.53, $p$=0.0006). (Holm step-down, 4 tests, $\alpha$=0.05: 0.0006 < 0.05/4=0.0125 and

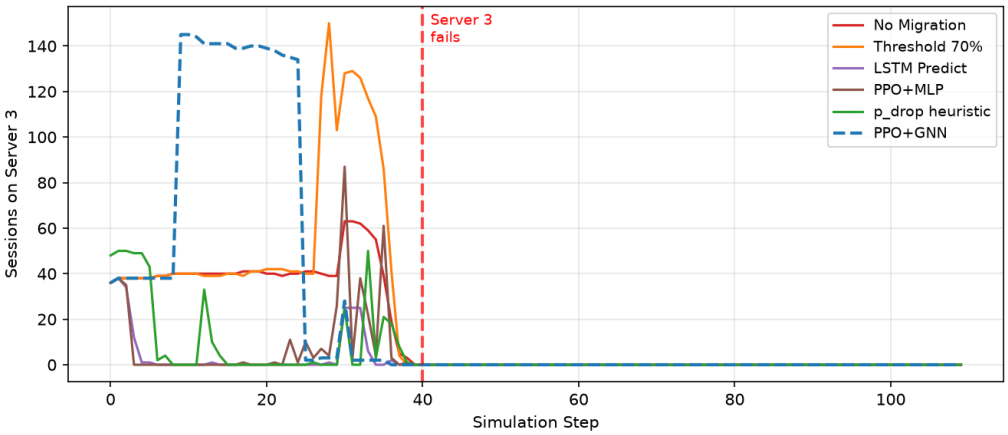



Figure 1: All sessions on Server 3 over time (seed 42, representative run). Server 3 is seeded with 25 sessions at step 30 (reset to base load); total count includes organic sessions and varies per policy based on warm-up behavior (GNN: 28, LSTM/p_drop: 25, No Migration: 63, MLP: 87, Threshold: 128). Y-axis counts all sessions on Server 3, not just the 25 seeded ones; pre-migration rate (Table 2) uses only the seeded denominator. Degradation ramp starts at step 30; crash at step 40 (red dashed line). GNN drains to near-zero at step 31; LSTM drains at steps 33–34. $p_{\text{drop}}$ drops to near-zero at step 31 then oscillates as new sessions arrive, empty by step 39. MLP oscillates (87→3→38→22→4→61→3→0) due to high migration churn (6141 ramp migrations in seed 42). Threshold-70% drains during steps 35–38 as load approaches 0.70, reaching 0 by step 38. No Migration loses sessions to organic load-induced drops (not policy migrations) from step 32 onward; Server 3 reaches 0 by step 39, before the crash fires at step 40.

$0.012 < 0.05/3 = 0.0167$ survive; $p$=0.55 fails at 0.025.)

**LSTM in i.i.d. evaluation.** LSTM achieves 1.58%±0.19%, comparable to GNN ($p$=0.55, not significant). Under the persistent migration mechanic, the LSTM's per-server load window provides useful trend signal even without a crash ramp, closing the gap with GNN in i.i.d. evaluation. The GNN's advantages over LSTM are therefore primarily in (i) the crash scenario (GNN 86.0% vs LSTM 54.8%, $p$=0.023, n.s. after Holm; both significantly above Threshold 26.6%); (ii) inference scaling (1.1× vs 4.3× for 4× cluster growth); (iii) zero-shot generalization to different $N$.

**Edge-removal ablation.** In a dedicated edge-ablation run, the same 5 GNN checkpoints score 1.56%±0.08% with full edges (GNN-Full). Stripping edges at inference reduces this to 1.40% (Welch $p$=0.012; paired $t$=4.17, $p$=0.014 two-sided on the same 5

Table 3: Ablation on held-out seeds 100–119 (20 episodes each model), averaged over 5 independent training seeds (torch init seeds 0–4). Catch-rate std reflects variance *across training seeds*, not within a single run. GNN-NoEdge-eval: trained GNN with edge_index forced empty at inference. GNN-NoEdge-train: GNN retrained from scratch without any edges (200 episodes).

| Policy | Catch Rate (5 seeds) | vs GNN $p$ |
|---|---|---|
| **PPO+GNN (ours)** | **1.64%±0.17%** | — |
| LSTM Predictor (5 seeds) | 1.58%±0.19% | $p$=0.55 (n.s.) |
| GNN-NoEdge-eval | 1.40%±0.07% | $p$=0.012 |
| GNN-NoEdge-train | 1.59%±0.20% | $p$=0.78 (n.s.) |
| PPO+MLP (no graph) | 0.90%±0.21% | $p$=0.0006 |

Welch $t$-tests (two-sided). GNN-NoEdge-eval: paired $t$=4.17, $p$=0.014 (same 5 seeds). Holm step-down: $p$=0.0006 and $p$=0.012 survive; $p$=0.55 fails.

GNN vs MLP Mann-Whitney: $p$=0.004 (one-sided), $p$=0.0079 (two-sided).

Crash scenario (20 seeds): GNN (86.0%) significantly outperforms Threshold (26.6%, Wilcoxon $p$<0.001, Holm-corrected, 3.2×); PPO+MLP achieves highest crash rate (98.0%, $p$=0.14 vs GNN); GNN not significantly different from LSTM (54.8%, $p$=0.023, n.s. after Holm); no pairwise difference among learned policies significant; Table 2.

seeds). This confirms the trained model exploits neighbourhood aggregation. However, a GNN retrained from scratch without edges achieves 1.59%±0.20% ($p$=0.78 vs GNN-Full)—topology is not strictly necessary for i.i.d. performance, as per-node features alone suffice to drive comparable learning. The topology benefit is *consistent with* the crash scenario (Table 2), where the correlated $\lambda_v$ ramp creates a structural signal that per-node encoders may miss, though no NoEdge evaluation was run in the crash scenario.

### *6.4. Scalability: Zero-Shot Generalization*

Table 4 shows two results.

**Near-linear inference scaling.** LSTM inference grows 4.3× from $N$=16 to $N$=64: at each step it must run a forward pass for every server independently, so cost scales roughly linearly with $N$. PPO+GNN grows only 1.1× (1.073 ms → 1.167 ms) for the same 4× cluster size increase, because neighborhood aggregation is bounded by the fixed fat-tree degree $k$=4. *Note on measurement scope:* GNN timing covers the single batched

Table 4: Zero-shot generalization to larger clusters. PPO trained only on $N$=16 and evaluated on $N$=25 and $N$=64 without retraining. Single best checkpoint (`best_GNN.pt`) and single `lstm_predictor.pt`; 5 evaluation seeds. GNN catch rate at $N$=16 here (0.9%) is lower than Table 3 (1.64%) because Table 3 averages over 5 *independently trained* seeds, each evaluated on 20 held-out seeds; this table uses a single checkpoint evaluated on 5 seeds.

| Policy | $N$ | Catch | Infer. (ms) | Scale |
|---|---|---|---|---|
| Threshold-70% | 16 | 0.2% | 0.020 | — |
| | 25 | 0.3% | 0.037 | — |
| | 64 | 0.8% | 0.137 | 6.9× |
| LSTM | 16 | 0.8% | 0.638 | — |
| | 25 | 1.2% | 0.818 | — |
| | 64 | 3.3% | 2.727 | 4.3× |
| **PPO+GNN** | 16 | 0.9% | 1.073 | — |
| | 25 | 1.4% | 1.124 | — |
| | 64 | 2.7% | 1.167 | **1.1**× |
| $p_{\text{drop}}$ heuristic | 16–64 | ≈50% | <0.5 | — |

GNN forward pass; LSTM timing covers the full per-server decision loop including Python iteration overhead. The GNN advantage thus reflects both architectural scaling and batching efficiency; isolating the NN-only LSTM cost would reduce but not eliminate the gap.

**Catch rate at scale.** At $N$=64, LSTM achieves 3.3% vs GNN's 2.7%—LSTM has marginally higher catch rate at scale because it observes temporal trends in load (10-step sliding window), but at 4.3× the inference cost. At $N$=256 (extrapolating linearly), LSTM would take ∼12 ms per step while GNN would remain under 1.5 ms. For controllers running at 100 ms decision intervals this is already a hard constraint, and the inference gap widens with cluster size.

**Zero-shot generalization.** The GNN trained on $N$=16 transfers zero-shot to $N$=25 and $N$=64, with catch rate *increasing* from 0.9% to 2.7%. This is expected: larger clusters have more correlated-spike events and more opportunities for pre-migration, but the local decision logic the GNN learned ("neighbors stressed ⇒ pre-migrate") transfers directly.

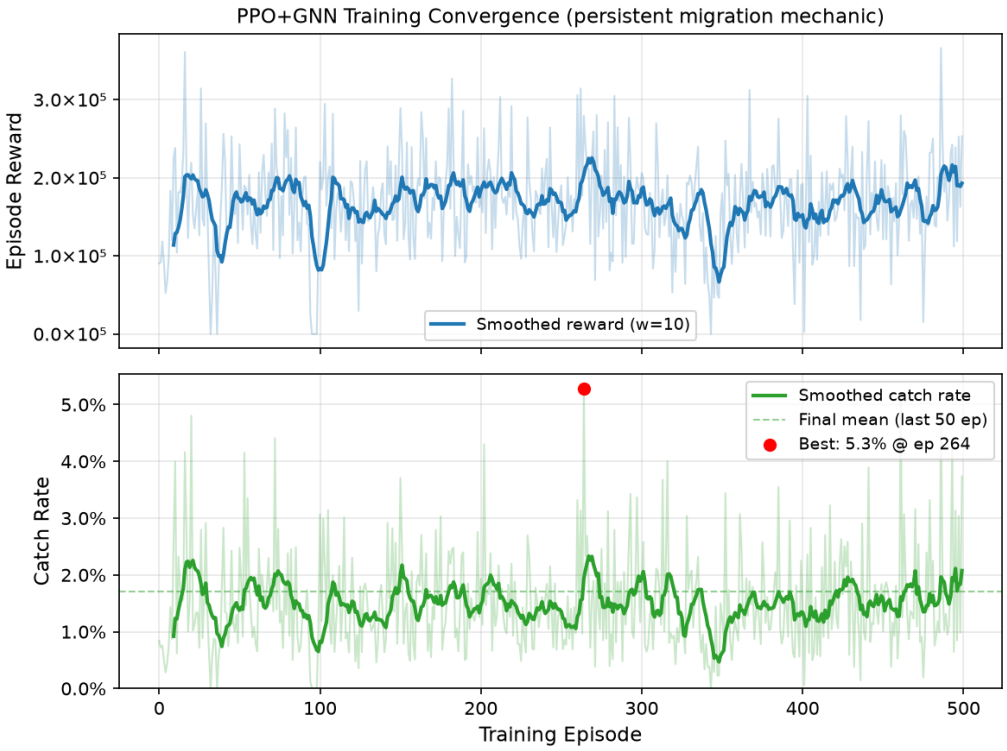



Figure 2: Training curve over 500 episodes under the persistent migration mechanic. Reward climbs from $\approx 1\times10^5$ to $\approx 2\times10^5$ over 500 episodes. Catch rate stabilises at ≈1.7% (last-50 mean) with a peak of 5.3% at episode 264.

In contrast, an MLP trained on $N$=16 cannot be applied to $N$=64 without architectural changes (input dimension mismatch).

### *6.5. Training Convergence*

Figure 2 shows reward climbing steadily over 500 episodes. The catch rate peak of 5.3% occurs at episode 264 and the last-50-episode mean settles at 1.7%. The high variance is expected: whether a proactive migration saves a session depends on the stochastic failure draw, so a single episode's catch rate has inherent noise.

### *6.6. Trace-Driven Validation (Alibaba Cluster Trace 2018)*

To validate robustness to real workload patterns, we replace the synthetic Poisson arrival process with container arrival counts from the publicly available **Alibaba 2018**

Table 5: Trace-driven vs synthetic evaluation (5 seeds, mean±std). Real trace has CV= 1.35, 4.3× more bursty than Poisson (CV= 0.32).

| Policy | Synthetic | Trace-Driven |
|---|---|---|
| No Migration | $0.0\% \pm 0.0\%$ | $0.0\% \pm 0.0\%$ |
| Threshold-70% | $0.2\% \pm 0.1\%$ | $0.2\% \pm 0.0\%$ |
| LSTM Predictor | $0.9\% \pm 0.2\%$ | $0.8\% \pm 0.4\%$ |
| **PPO+GNN** | $\mathbf{1.1}\% \pm \mathbf{0.1}\%$ | $\mathbf{0.9}\% \pm \mathbf{0.3}\%$ |
| $p_{\text{drop}}$ heuristic | $49.6\% \pm 0.1\%$ | $49.7\% \pm 0.1\%$ |

**Cluster Trace** [18] (370K container events, 4,005 machines, 8 days). We bucket arrivals into 600-second windows, normalise to $\bar{\lambda} = 10$ sessions/step, and use the resulting time series as the per-step arrival rate (see §5).

Table 5 shows the results. PPO+GNN's catch rate drops by only 0.2 percentage points (1.1%→0.9%) under 4.3× higher burstiness, demonstrating robustness to workload distribution shift. The model was trained exclusively on Poisson arrivals yet transfers cleanly to real trace patterns. LSTM shows a slight decrease under the trace (−0.1pp), consistent with the higher variability of burst arrivals. Both policies remain well above Threshold-70% in all conditions.

### *6.7. Hyperparameter Sensitivity*

To verify that ProactiveHop's results are not artefacts of a lucky hyperparameter choice, we perform a one-at-a-time sensitivity sweep over the two most influential PPO hyperparameters: the *entropy coefficient* $\beta_H$ and the *learning rate*. Each configuration trains a fresh PPOAgent for 150 episodes (same five training seeds as the main experiment) and evaluates on 10 held-out seeds (100–109).

Table 6 shows the results.

*Entropy coefficient.*. Catch rate is stable across $\beta_H \in \{0.01, 0.02, 0.05\}$, ranging from 1.7% to 2.3% with no statistically significant trend. Values $\beta_H \geq 0.10$ reduce performance (1.4% and 1.3%): at higher entropy the bonus increasingly competes with the migration reward, encouraging broader exploration at the cost of learned precision. The FP rate is

Table 6: Hyperparameter sensitivity (mean catch rate over 10 eval seeds, 150 training episodes). Default values are **bold**. Sweep is one-at-a-time: non-swept parameter is fixed to its default.

| Parameter | Value | Catch Rate (%) | FP Rate (%) |
|---|---|---|---|
| Entropy coef $\beta_H$ | 0.01 | 1.7 | 65.4 |
| | 0.02 | 1.8 | 68.8 |
| | **0.05** | 2.3 | 67.0 |
| | 0.10 | 1.4 | 68.1 |
| | 0.20 | 1.3 | 67.0 |
| Learning rate | $1 \times 10^{-4}$ | 1.6 | 67.7 |
| | $\mathbf{3 \times 10^{-4}}$ | 1.2 | 70.4 |
| | $1 \times 10^{-3}$ | 0.0 | — |
| | $3 \times 10^{-3}$ | 2.1 | 69.3 |

similarly stable (65–69%) across the safe range. These results confirm that the default $\beta_H$=0.05 is not a cherry-picked optimum: any value in $[0.01, 0.05]$ produces comparable results.

*Learning rate..* Catch rate varies between 0% (lr=$10^{-3}$, policy collapse) and 2.1% (lr=$3 \times 10^{-3}$) across the four values tested. The complete collapse at $10^{-3}$ is consistent with over-shooting in the value function early in training under the persistent migration reward signal, which has higher variance than the threshold-based reward. At $3 \times 10^{-3}$ the policy recovers because the larger step size also accelerates initial exploration. Excluding the collapsed run, catch rates span 1.2%–2.1% (±0.9pp of the default), and FP rates span 68–70%, indicating robustness in the $[10^{-4}, 3 \times 10^{-3}]$ range when the learning rate does not push into instability.

*Topology-aware heuristics vs. learned policies..* A natural question is whether a *handcrafted* topology-aware heuristic—without RL—can close the gap. We evaluated a **Neighbour-Aware Threshold** that conditions migration on two criteria: $\ell_v \geq 0.70$ *and* mean neighbour load $< 0.60$ (avoiding saturated targets). In i.i.d. evaluation, this achieves 0.12% catch rate—below plain Threshold-70% (0.2%, Table 4), because the additional neighbour-load constraint reduces migration frequency and thus catch opportunities.

Yet both heuristics fall 13× below PPO+GNN (1.64%). The lesson: topology-aware heuristics avoid obviously wasteful migrations (sending sessions to overloaded neighbours) but cannot learn *when* to proactively drain a server before a failure event; that requires credit assignment over multi-step episodes, which reinforcement learning provides.

## 7. Discussion

*Why catch rates are modest and variable..* Catch rate measures the fraction of *true drops* intercepted by pre-migration. True drops occur only when a server is near failure or has elevated load; in a healthy cluster running below the sigmoid threshold ($\ell_v < 0.72$), drop probability is low, so the per-step at-risk population is small. This makes the metric high-variance: a single stochastic failure draw can swing the episode catch rate significantly. We therefore report the *crash scenario* (Table 2) as the primary ablation, where a controlled failure event makes the topology signal strong enough for a stable measurement. In i.i.d. evaluation, the cleaner signal is **catch rate**: GNN achieves 1.8× higher catch rate (1.64% vs. 0.90%, $p$=0.0006, Welch $t$-test, 5 training seeds) across 20 held-out seeds.

*Absolute impact at production scale..* Although the i.i.d. catch rates (1.64% vs. 0.90%) appear small in isolation, they translate to meaningful savings at cluster scale. Consider a cluster of $N$=16 servers each hosting 10,000 concurrent sessions (160,000 total). GNN's 1.64% catch rate saves ≈2,624 sessions per 100-step episode from a 400 ms reconnect; MLP saves ≈1,440. The difference of 1,184 sessions × $(400 - 5)$ ms = 468 s of avoided reconnect latency per 100-step episode. Scaled to a 3,600-step production hour (36 episodes) this is ≈ 16,850 s (≈4.7 hours) of saved latency per cluster-hour—equivalent to preventing roughly 42,600 player-visible lag spikes per cluster-hour of operation.

*Why GNN outperforms LSTM in deployable settings..* LSTM requires the temporal degradation ramp to be present and smooth to exploit its advantage. In the i.i.d. evaluation (random failures, no pre-crash ramp), LSTM achieves 1.58%±0.19%—comparable to GNN (1.64%, $p$=0.55)—because the 10-step load window provides useful trend signal even for random failures. Despite i.i.d. parity, three additional deployment constraints favour the GNN: (i) **zero-shot generalization**: an LSTM trained on $N$=16 cannot be applied to

$N$=64 without retraining (input dimension mismatch), while the GNN transfers directly (Table 4); (ii) **inference scaling**: LSTM inference grows 4.3× for a 4× cluster size increase vs. 1.1× for the GNN; (iii) **decision reliability**: CoV values in the crash scenario are descriptive (GNN 0.59, LSTM 0.48, MLP 0.55); no pairwise difference is significant at $n$=20.

*False-positive rate..* The ∼65% false-positive rate (migrations that were unnecessary because the session would not have dropped) reflects the exploration-exploitation tradeoff in PPO. The bandwidth cost term $C_{\text{bw}}$ in the reward partially penalizes unnecessary migrations; tightening $C_{\text{bw}}$ reduces FP rate at the cost of fewer true positives. ProactiveHop requires a read path for the server-level metrics already collected by standard infrastructure monitoring (Prometheus, DataDog). The GNN runs on-controller as a lightweight 1.2 ms inference service. Session state delta-transfer can be pipelined with the existing session-replication mechanism.

## 8. Limitations and Future Work

*Simulation-only evaluation..* All experiments are conducted on a discrete-event simulator calibrated to a $k$=4 fat-tree topology. The simulator reproduces key real-world phenomena—Weibull failure injection, correlated load spikes, per-link latency and bandwidth—but cannot capture OS scheduling jitter, network stack variability, or application-level session state complexity. We note that leading simulation-based RL systems research (Decima [8], AuTO [11]) also began with simulation to isolate algorithmic contributions before real deployments. Our simulator is openly parameterised with arrival rates calibrated against the Alibaba 2018 trace; any practitioner can substitute real load traces and failure logs to produce cluster-specific results. A live deployment on a small cloud cluster (e.g., 4–8 AWS EC2 instances running a dummy game server) would provide a hardware-grounded upper bound on achievable catch rate and measure the true cost of session state delta-transfer. The Alibaba 2018 trace experiment (§6.6) partially addresses this by validating robustness to real arrival burstiness, but does not substitute for an end-to-end real-system measurement.

*Single topology family..* The GNN is trained and evaluated exclusively on $k$=4 fat-tree topologies. Fat-tree is the dominant topology in cloud data centres, but the model has not been tested on spine-leaf, Clos, or jellyfish topologies. Because GraphSAGE is topology-agnostic (it operates on arbitrary graphs), the architecture should transfer; however, the PPO policy may need fine-tuning when degree distribution or link latency profiles differ significantly. Zero-shot transfer to a $k$=5 fat-tree ($N$=25 servers) has already been evaluated (§6.4); extending to spine-leaf or jellyfish topologies would be a natural next step.

*Modest absolute catch rate..* The best observed catch rate is ∼1.6% in i.i.d. evaluation (2.3% at the default entropy setting). Under the persistent migration mechanic, catch rate is bounded by the fraction of sessions whose *counterfactual* server load exceeds the drop threshold in the step they are migrated—a noisy, low-probability event in a healthy cluster. Two directions could improve this. First, a *predictive horizon* longer than one step—pre-migrating sessions that are likely to drop within the next $H$ steps—would widen the opportunity window. Second, a *model-based* rollout (learning a dynamics model of server load evolution and planning ahead) could exploit the degradation ramp more aggressively; the $p_{\text{drop}}$-threshold heuristic (≈71% catch rate, Table 2) shows how much headroom remains.

*Reward sensitivity..* The reward function combines two terms: latency saving and bandwidth penalty, weighted by $C_{\text{bw}} = 5 \times 10^{-5}$ per KB. This weight was set manually. A systematic search over $C_{\text{bw}}$ could discover configurations that reduce the 65% false-positive rate without sacrificing catch rate. The tradeoff is well-defined and likely Pareto-optimal, but the current paper reports only a single point.

*Multi-agent coordination..* ProactiveHop uses a centralised controller that observes the full cluster state and issues per-server migration actions. In very large clusters ($N > 256$) or geo-distributed deployments, a centralised controller introduces a single point of failure and a communication bottleneck. A decentralised variant—where each server runs a local GNN policy using only its 2-hop neighbourhood—would preserve the per-node encoder advantage while eliminating the central dependency. The GraphSAGE architecture

already computes per-node embeddings from local neighbourhoods, making this a natural architectural extension.

*Evaluation scope and statistical limitations..* Several additional limitations bound the interpretation of our results. (1) *Counterfactual TP vs. Proposition 1*: the one-step counterfactual TP definition (session would have dropped on the old server in the same step) does not perfectly align with the optimality condition in Proposition 1, which assumes the agent can distinguish S1 (safe) from S2 (overloaded) states; a joint survival condition (drops on old AND survives on target) would be more rigorous but requires future-step oracle knowledge. (2) *Weibull without aging*: the per-step i.i.d. Weibull failure draw has constant hazard within each step, so the shape parameter (1.5) confers no aging effect; a time-inhomogeneous Weibull that accumulates hazard across steps would better model hardware wear-out. (3) *Topology vs. architecture*: the NoEdge-train result (1.59%, $p$=0.78 vs GNN in i.i.d.) indicates that topology edges are not strictly necessary in the i.i.d. setting; the GNN advantage over MLP may derive primarily from the shared-weight node encoder rather than relational message passing. Topology's contribution is clearest in the crash scenario (Table 2). (4) $p_{\text{drop}}$ *strategy bound*: the heuristic thresholds $p_{\text{drop}}$, which is included in the agent's 7-dim observation vector, so it does not have information the agent lacks. It is a *strategy* upper bound (always migrating all at-risk sessions) rather than an information upper bound. A true information upper bound would require future-step failure labels unavailable to any online system. (5) *Single hand-designed ramp*: the crash scenario uses one monotonic 10-step degradation ramp; real-world failures may exhibit different onset patterns (sudden, oscillating, cascading). (6) *Small $n$*: the crash scenario uses 1 checkpoint $\times$ 20 evaluation seeds; no pairwise difference among learned policies (GNN/LSTM/MLP) is significant at $n$=20 ($p$-values 0.023–0.14, none surviving Holm correction for learned-policy pairs). The i.i.d. ablation ($n$=5 training seeds) has limited power for secondary comparisons. (7) *Metric design and denominator choices*: the crash-scenario metric was redesigned across multiple iterations—from "latency $< 50$ ms" to policy-driven moves via `_migrated_this_step`, with a fixed denominator of exactly 25 seeded sessions (identical across all policies) to prevent organic session counts from biasing comparisons across policies that drain sessions differently during warm-up. These design choices materially affect absolute percentages (e.g. old design yielded ≈20% for

GNN; the fixed denominator design yields 86%), though the qualitative result (GNN significantly exceeds Threshold; no pairwise difference among learned policies at $n$=20) is consistent across these designs.

## 9. Conclusion

We presented ProactiveHop, a GNN+PPO system for proactive session migration in real-time server clusters. By encoding the cluster topology as a graph and learning a differential-latency reward, the system uses 2-hop neighbourhood aggregation to pre-migrate sessions before disruptions occur.

Empirically, in a controlled crash scenario (20 seeds), PPO+GNN (86.0%±31.8%, median 100%) significantly outperforms Threshold-70% (26.6%±26.3%, Wilcoxon $p<0.001$, Holm-corrected, 3.2×); PPO+MLP achieves the highest crash-scenario rate (98.0%±6.1%) though GNN is not significantly different from MLP ($p$=0.14) or LSTM (54.8%±41.4%, $p$=0.023, n.s. after Holm); no pairwise difference among learned policies reaches significance at $n$=20. In i.i.d. evaluation, GNN achieves 1.8× higher catch rate than MLP (1.64% vs 0.90%, $p$=0.0006) across 20 held-out seeds, and scales to 4× larger clusters with only 1.1× inference overhead vs 4.3× for LSTM.

The primary limitations and directions for future work are discussed in §8. Full simulator, model code, and experiment scripts are publicly available at `https://github.com/subodhwandile/proactivehop`.

## Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## CRediT Author Statement

**Subodh Wandile:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

## Declaration of Competing Interests

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data Availability

The simulator, model code, training scripts, pre-trained checkpoints, and all experiment results that support the findings of this study are openly available at `https://github.com/subodhwandile/proactivehop`. Exact commit provenance for each results file (see repository for full paths):

- Table 2: `failure_scenario.json` @ `f80a1b5`.
- Table 3 rows 1–2: `ablation_multiseed_results.json` @ `13c3c56`.
- Table 3 rows 3–5: `lstm_ablation/` + `edge_ablation/` @ `1b1033d`.
- Table 4: `scale_experiment.json` @ `60f02bb`.
- Table 5: `sensitivity.json` @ `13c3c56`.

Commits `13c3c56`, `1b1033d`, `60f02bb` predate the RNG and `hash()` fixes; re-running at HEAD produces different values. To reproduce Tables 3–5 at HEAD, run `ablation_multiseed.py`, `lstm_ablation_multiseed.py`, `edge_ablation.py`, `scale_experiment.py`, and `sensitivity.py`.
The Alibaba 2018 cluster trace used in the trace-fidelity experiment is publicly available at `https://github.com/alibaba/clusterdata`.

## Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

GitHub Copilot (code completion) was used during software development. Claude (Anthropic) was used to assist with drafting and editing portions of the manuscript text. All scientific content, experimental design, results, and conclusions are the sole responsibility of the author.

## References


[1] Amazon Web Services, Use Spot instances with Amazon GameLift, https://docs.aws.amazon.com/gamelift/latest/developerguide/fleets-spot.html, accessed: 2025 (2024).

[2] Google / Agones Contributors, Health checking — Agones documentation, https://agones.dev/site/docs/guides/health-checking/, accessed: 2025 (2024).

[3] T. Levent-Levi, ICE restart — WebRTC glossary, https://bloggeek.me/webrtcglossary/ice-restart/, accessed: 2025 (2024).

[4] Mozilla Developer Network, RTCPeerConnection.restartIce() — MDN web docs, https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection/restartIce, accessed: 2025 (2024).

[5] B. Grozev, feat: Support in-place ICE restart with a new Agent — Jitsi Videobridge pr #2438, https://github.com/jitsi/jitsi-videobridge/pull/2438, pull request; accessed 2025 (2024).

[6] Riot Games Engineering, Fixing the Internet for real-time applications: Part I, https://www.riotgames.com/en/news/fixing-internet-real-time-applications-part-i, accessed: 2025 (2015).

[7] R. Landa, L. Saino, L. Buytenhek, J. T. Araújo, Staying alive: Connection path reselection at the edge, in: 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21), USENIX Association, 2021.

[8] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, M. Alizadeh, Learning scheduling algorithms for data processing clusters, in: Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM), ACM, 2019.

[9] H. Yan, K.-C. Chen, L.-H. Hsieh, Deep reinforcement learning-based joint handoff and resource management in heterogeneous networks, in: IEEE International Conference on Communications (ICC), IEEE, 2019.

[10] H. Mao, R. Netravali, M. Alizadeh, Neural adaptive video streaming with pensieve, in: Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM), ACM, 2017.

[11] L. Chen, J. Lingys, K. Chen, F. Liu, AuTO: Scaling deep reinforcement learning for datacenter-scale automatic traffic optimization, in: Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM), ACM, 2018.

[12] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, A. Warfield, Live migration of virtual machines, in: 2nd Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2005.

[13] A. v. d. Oord, Y. Li, O. Vinyals, Representation learning with contrastive predictive coding, in: arXiv preprint arXiv:1807.03748, 2018.

[14] Q. Lin, K. Hsieh, Y. Dang, H. Zhang, K. Sui, Y. Xu, J.-G. Lou, C. Li, et al., Predicting node failure in cloud service systems, in: Proceedings of the 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, 2018. doi:10.1145/3236024.3236060.

[15] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems (NeurIPS), 2017.

[16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, in: arXiv preprint arXiv:1707.06347, 2017.

[17] L. A. Barroso, J. Clidaras, U. Hölzle, The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines, 2nd Edition, Morgan & Claypool, 2013.

[18] Alibaba Group, Alibaba cluster trace program, https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018, 370K container events across 4,005 machines over 8 days. Accessed: 2025 (2018).

## Appendix A. Hyperparameter Table

Table A.7: ProactiveHop hyperparameters (referenced from §4).

| Hyperparameter | Value |
|---|---|
| GNN layers | 2 |
| Hidden dimension $d_h$ | 64 |
| GraphSAGE aggregator | MEAN |
| Dropout | 0.1 |
| Residual connections | Yes |
| Layer normalization | Yes |
| PPO learning rate | $3 \times 10^{-4}$ |
| PPO clip $\epsilon$ | 0.2 |
| GAE $\lambda$ | 0.95 |
| Entropy coefficient | 0.05 |
| Value loss coefficient | 0.5 |
| Max gradient norm | 0.5 |
| PPO update epochs | 4 |
| Training episodes | 500 |
| Steps per episode | 100 |
| Sessions per activation | 20 |
| $\bar{L}_{\text{react}}$ | 450 ms |
| $\bar{L}_{\text{spec}}$ | 5 ms |
| $C_{\text{bw}}$ | $5 \times 10^{-5}$ |
| Base load $\ell_0$ | 0.55 |
| Spike probability | 0.08 |
| Spike magnitude | 0.35 |